Universidad del Valle de Guatemala

Facultad de Ingeniería
Departamento de Ciencias de la Computación

Computación Paralela y Distribuida
Sección 20, Semestre II 2026



Proyecto 2

Búsqueda de llave DES con MPI

Roberto Barreda, 23354
Nina Najera, 231088
José Antón, 221041

Guatemala, 13 de octubre de 2026

Índice

1. Introducción **1**

2. Antecedentes de DES y fuerza bruta **1**

3. Cifrado y rutinas principales **1**

4. Comunicación y sincronización **3**

5. Acercamientos de búsqueda **5**

5.1. Versión secuencial 5

5.2. Rangos contiguos o naive 5

5.3. Distribución cíclica 5

5.4. Recorrido permutado 5

5.5. Bloques dinámicos 5

5.6. Versión híbrida 6

6. Pruebas de cifrado y posición de la llave **6**

6.1. Distintos tamaños del espacio secuencial 7

7. Evaluación de tiempos y speedup **7**

7.1. Condiciones de medición 7

7.2. Comparación con cuatro procesos 8

7.3. Cambio en la cantidad de procesos 8

7.4. MPI con OpenMP 8

8. Conclusiones **12**

9. Recomendaciones **13**

A. Catálogo de funciones y librerías **14**

B. Bitácora y capturas de ejecución **17**

1. Introducción

Se desarrolló un programa en C que recupera una llave DES por fuerza bruta. Para reconocer una posible solución, se descifra el archivo con cada llave y se busca una frase conocida dentro del resultado. Se implementaron una versión secuencial, una versión MPI con rangos contiguos y tres alternativas de reparto: cíclica, permutada y por bloques dinámicos. También se incluyó una versión híbrida con MPI y OpenMP.

Se compararon los tiempos con distintas llaves y cantidades de procesos. El análisis distingue entre una mejora por repartir el trabajo y una búsqueda que termina pronto por la posición favorable de la llave. Se registraron los comandos, las salidas y las mediciones para comprobar los resultados.

El código y los archivos del proyecto se encuentran en el repositorio: https://github.com/Ninaswiftie09/Computacion_Paralela/tree/Proyecto2.

2. Antecedentes de DES y fuerza bruta

DES es un cifrador simétrico que trabaja con bloques de 64 bits. Su llave contiene 56 bits útiles y 8 bits de paridad. Por eso existen 2^{56} llaves posibles, aproximadamente $7{,}2 \times 10^{16}$ [1]. El programa representa los bits útiles con un entero sin signo y agrega la paridad antes de llamar a OpenSSL.

El cifrado comienza con una permutación de los bits del bloque y los divide en dos mitades de 32 bits. A partir de la llave se preparan 16 subllaves de 48 bits. En cada una de las 16 rondas, la mitad derecha se expande, se combina con la subllave mediante XOR y pasa por las cajas de sustitución y una permutación. Su resultado se combina con la mitad izquierda. Las mitades cambian de posición y, al terminar las rondas, se aplica la permutación final.

El descifrado usa la misma estructura con las subllaves en orden inverso. En este proyecto se usa el modo ECB, que procesa cada bloque de forma independiente. El archivo se completa con ceros cuando su longitud no es múltiplo de ocho bytes. Se emplea la biblioteca OpenSSL como reemplazo de la interfaz rpc/des_crypt.h [2].

La fuerza bruta prueba llaves hasta encontrar una que produzca la frase buscada. Una frase larga reduce la posibilidad de aceptar una llave incorrecta. La presencia de la frase es un criterio de reconocimiento, por lo que el resultado se presenta como una llave posible. Para las pruebas se verificó además que coincidiera con la llave usada al cifrar.

3. Cifrado y rutinas principales

Se carga el texto desde un archivo y se conserva una copia del cifrado durante toda la búsqueda. La llave se divide en ocho grupos de siete bits y se ajusta la paridad impar de cada byte. OpenSSL genera las subllaves y realiza las operaciones DES sobre cada bloque.

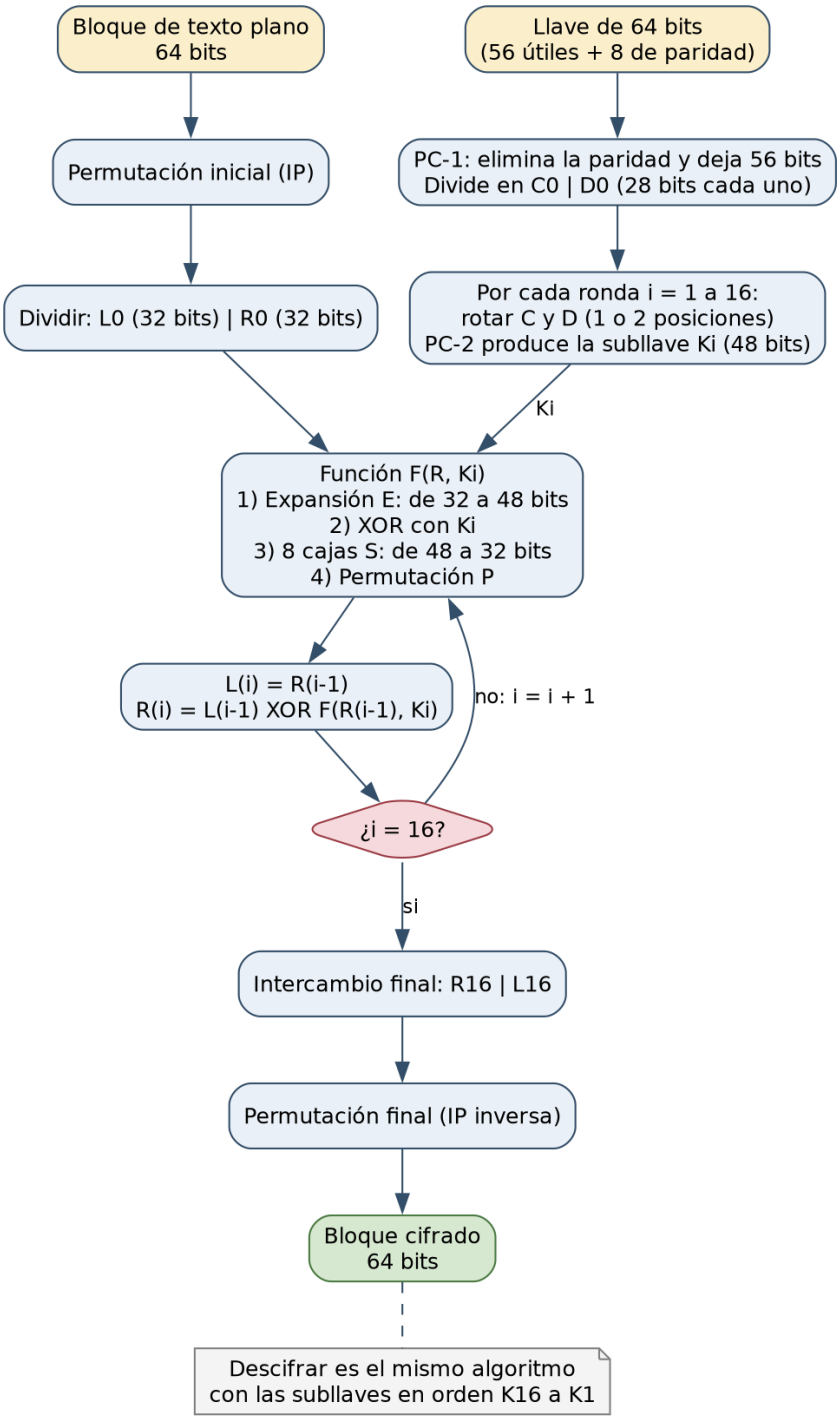


Figura 1: Pasos principales del cifrado DES. El descifrado invierte el orden de las subllaves.

Las funciones des_cifrar y des_descifrar corresponden a encrypt y decrypt. Reciben la llave, el arreglo de bytes y su longitud. Ambas modifican el mismo arreglo y requieren una longitud múltiplo de ocho. La operación elegida determina si se cifra o se descifra.

La función des_probar_llave corresponde a tryKey. Primero copia el cifrado con memcpy a un arreglo de trabajo. Luego lo descifra, agrega un cero al final y usa strstr para buscar la frase. memcpy copia una cantidad indicada de bytes. strstr devuelve la posición de la primera coincidencia o un puntero nulo cuando no existe. La copia evita que un intento cambie los datos de los siguientes intentos.

El programa valida el archivo, el rango de la llave, los bits de búsqueda y la frase. Rechaza archivos vacíos, archivos mayores de un MiB y cifrados que no tengan una longitud múltiplo de ocho. Se reserva memoria antes de buscar y se libera al terminar. Cada hilo del híbrido tiene su propio arreglo de trabajo. Se verificaron 108 casos MPI y cinco recuperaciones secuenciales. Se incluyeron llaves en los extremos del espacio, búsquedas sin solución, bloques pequeños, frases inválidas y archivos incorrectos.

4. Comunicación y sincronización

El proceso 0 valida los argumentos y carga el archivo. Luego comparte el resultado de la validación, el cifrado y la frase mediante MPI_Bcast. Si un proceso no puede reservar memoria, se acuerda la salida antes de iniciar la búsqueda.

MPI_Irecv deja preparada una recepción sin detener el recorrido de llaves. MPI_Test revisa cada 1024 intentos si llegó un aviso. El proceso que encuentra una llave envía el aviso con MPI_Isend y conserva el dato hasta completar el envío. Esta operación permite atender varios hallazgos sin depender del espacio interno de mensajes de MPI [3].

MPI_Wait completa la recepción pendiente. Antes de esperar, MPI_Allreduce cuenta los hallazgos. Si no se espera ningún aviso, la recepción se cancela y después se completa. Si existen varios hallazgos, se reciben también los avisos restantes y se cierran los envíos con MPI_Waitall. Se informa la menor llave hallada.

MPI_Send se usa en el reparto dinámico. Cada trabajador envía su estado y espera un rango del coordinador. El coordinador recibe un pedido y responde a ese trabajador. El orden pedido y respuesta evita que ambos queden esperando un envío que todavía no fue recibido.

Se usa una barrera MPI explícita para alinear el arranque del cronómetro. El tiempo paralelo es el mayor tiempo local de búsqueda. Incluye la detención de los procesos, pero excluye la lectura inicial y la reunión final de resultados.

En el híbrido, solo el hilo principal llama a MPI y se comprueba el soporte MPI_THREAD_FUNNELED. La bandera de detención es atómica y cada hilo escribe su propio contador. Se coloca una barrera explícita antes de que el hilo principal sume los contadores. No se usan ciclos de reparto ni reducciones OpenMP con barreras implícitas. La unión al cerrar una región parallel es obligatoria en OpenMP y no admite nowait [4].

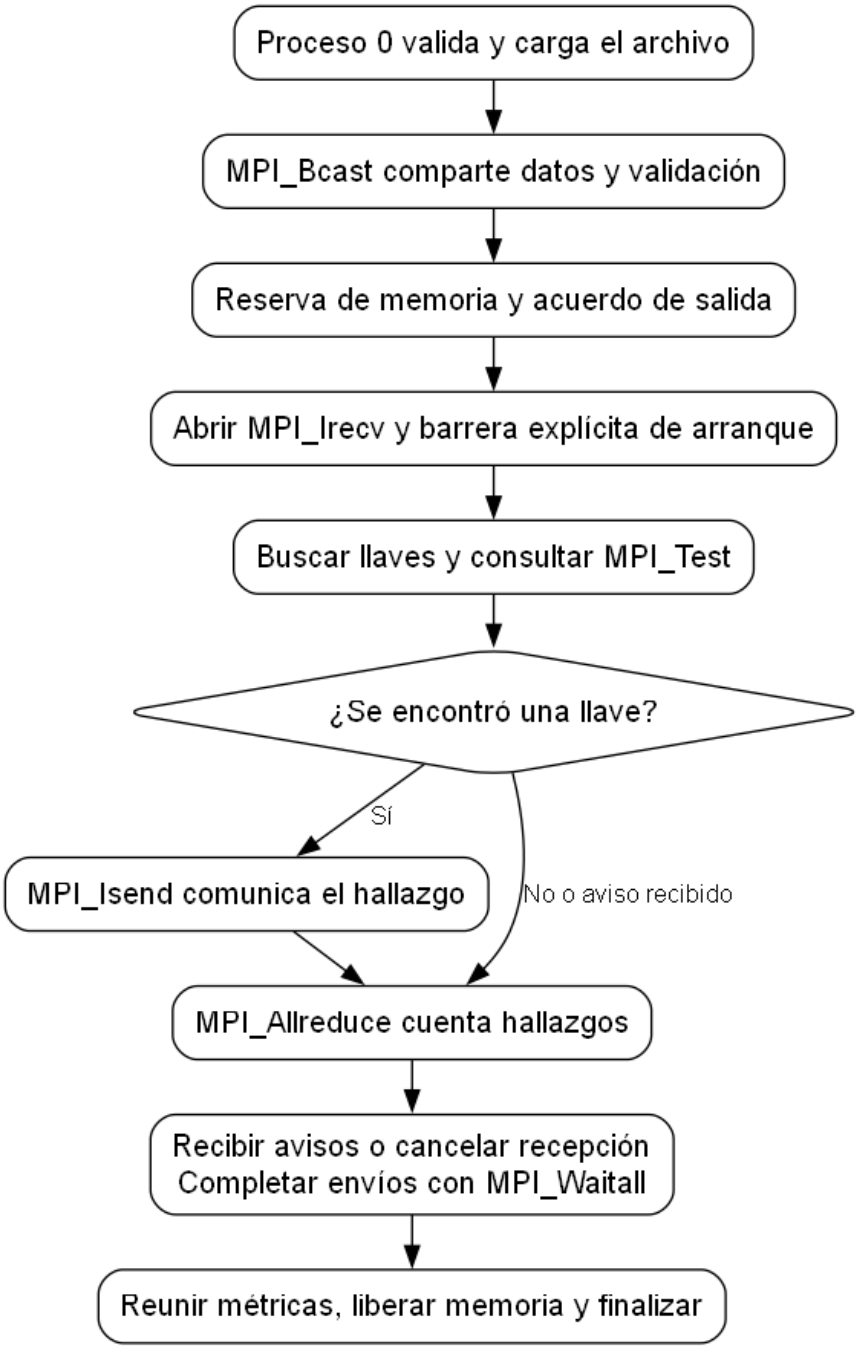


Figura 2: Preparación, búsqueda y cierre de la comunicación MPI.

5. Acercamientos de búsqueda

5.1. Versión secuencial

Se prueban las llaves desde cero hasta encontrar una coincidencia o agotar el espacio. Para una llave k, se realizan $k + 1$ intentos cuando no hay coincidencias anteriores. Esta versión proporciona el tiempo de referencia para calcular el speedup.

5.2. Rangos contiguos o naive

Se divide el espacio S en P rangos contiguos. El proceso r comienza en $r\lfloor S/P \rfloor$ y avanza de uno en uno. El último proceso toma el residuo. El tiempo depende de la distancia entre la llave y el inicio del rango que la contiene. Por eso una llave grande puede encontrarse casi de inmediato y otra cercana puede tardar mucho.

5.3. Distribución cíclica

El proceso r prueba las llaves r, $r + P$, $r + 2P$ y las siguientes. Los rangos no se repiten y todos los procesos avanzan sobre el espacio en conjunto. El proceso ganador realiza cerca de $k/P + 1$ intentos. El tiempo todavía depende del valor de la llave, pero su proporción frente al secuencial suele ser más regular.

5.4. Recorrido permutado

Se cambia el orden de las llaves mediante

$$k(j) = (Aj + C) \text{ mód } 2^b,$$

donde A es impar y b es la cantidad de bits del espacio de búsqueda. La transformación es invertible, por lo que no repite ni omite llaves. Cada proceso toma índices separados por P. La semilla determina A y C y permite repetir el recorrido.

El objetivo es reducir la relación directa entre el valor de la llave y su posición de búsqueda. El tiempo de una corrida todavía puede variar bastante. Se cambiaron las semillas entre repeticiones para observar esa variación. Cinco semillas permiten una comparación inicial, pero no prueban que todas las llaves tengan el mismo tiempo esperado.

5.5. Bloques dinámicos

El proceso 0 mantiene el siguiente rango disponible y entrega bloques de 4096 llaves a los demás procesos. Cuando un trabajador termina, pide otro bloque. Los procesos que avanzan

más rápido reciben más trabajo. El coordinador no prueba llaves, por lo que con P procesos quedan $P - 1$ trabajadores. Con un solo proceso se usa el recorrido secuencial.

Cuando un trabajador encuentra la llave, informa el hallazgo en su siguiente pedido. El coordinador deja de repartir trabajo y responde con un rango vacío. Los bloques que ya estaban activos terminan antes de pedir de nuevo. Así, el trabajo adicional por trabajador queda limitado al resto de su bloque.

```
Coordinador:
  siguiente = 0
  mientras queden trabajadores activos:
    recibir pedido y estado de un trabajador
    si informa un hallazgo: dejar de repartir
    si queda trabajo y no hay hallazgo:
      enviar el siguiente bloque sin repetirlo
    en otro caso:
      enviar rango vacio y retirar al trabajador
Trabajador:
  enviar estado y pedir bloque
  mientras el rango recibido no este vacio:
    probar sus llaves hasta encontrar la frase
    enviar resultado y pedir otro bloque
```

Los bloques pequeños aumentan la cantidad de mensajes. Los bloques grandes reducen esos mensajes, pero aumentan el trabajo realizado después de un hallazgo. Se eligió un tamaño fijo para comparar los algoritmos sin cambiar este parámetro entre llaves.

5.6. Versión híbrida

Se combina MPI con OpenMP. Con P procesos y T hilos se tienen PT trabajadores. Cada uno recorre una secuencia cíclica distinta y utiliza su propio arreglo de descifrado. El uso de más hilos añade costos de creación, coordinación y acceso a recursos compartidos, por lo que se compara con MPI puro usando la misma cantidad total de trabajadores.

6. Pruebas de cifrado y posición de la llave

Se utilizó el texto “Esta es una prueba de proyecto 2” y la frase “es una prueba de”. Se comprobó el cifrado y descifrado con la llave 42. El cifrado también se comparó byte a byte con openssl enc usando los mismos bytes de llave y el modo DES ECB.

Las pruebas con cuatro procesos sobre el espacio completo usan las llaves 123456, $2^{54} - 1$ y 2^{54}. La última llave es la primera del rango del proceso 1 y se encuentra en un intento. La llave $2^{54} - 1$ queda al final del primer rango y no terminó durante el límite de 20 segundos. Se registra como un tiempo mayor de 20 segundos, sin tratar el corte como un tiempo de búsqueda completo.

La división utiliza rangos que comienzan en cero. Por eso el cambio entre el último elemento del primer rango y el primero del siguiente ocurre exactamente entre $2^{54}-1$ y 2^{54}. Las salidas de estas pruebas se conservan en resultados/guia.txt.

6.1. Distintos tamaños del espacio secuencial

Se midió la búsqueda de la última llave de espacios de 16, 20, 22 y 24 bits. Limitar el espacio no cambia DES, que sigue usando una llave efectiva de 56 bits. Solo permite terminar las mediciones en un tiempo razonable.

Tabla 1: Búsqueda secuencial de la última llave de cada espacio.

Bits	Llave	Intentos	Tiempo en segundos
16	65535	65536	0.0337
20	1048575	1048576	0.5148
22	4194303	4194304	2.2136
24	16777215	16777216	8.7168

7. Evaluación de tiempos y speedup

7.1. Condiciones de medición

Las mediciones se realizaron en un contenedor Debian 13 sobre Docker Desktop y WSL2, con un Intel Core i7-1165G7, cuatro núcleos y ocho hilos lógicos. Se utilizó GCC 14.2.0 con O2, Open MPI 5.0.7 y OpenSSL 3.5.7. Los procesos no se fijaron a núcleos. Se habilitó el uso de hilos lógicos para las pruebas de ocho procesos. El registro completo está en resultados/entorno.txt.

Se usó un espacio de $S = 2^{22}$ y se realizaron cinco repeticiones por caso. Se probaron 1, 2, 4 y 8 procesos para los buscadores MPI. El híbrido usó las combinaciones 1 por 2, 1 por 4, 1 por 8, 2 por 4 y 4 por 2 procesos por hilos. Las 550 corridas conservan sus salidas y una comprobación de la llave recuperada.

Las llaves se definieron con las mismas posiciones relativas para todos los algoritmos. Las etiquetas fácil, media y difícil describen su relación con el reparto naive de cuatro procesos, no una dificultad propia de DES.

Se calcula el speedup como $T_s/\overline{T_p}$, donde T_s es el promedio secuencial de la misma llave y $\overline{T_p}$ es el promedio paralelo. La desviación estándar muestra la variación entre las cinco corridas. En el permutado incluye también el cambio de semilla. No se mezclan los datos de esta serie con mediciones anteriores.

Tabla 2: Llaves del espacio reducido.

Caso	Definición	Llave
Fácil	$S/2 + 1$	2097153
Media	$S/2 + S/8$	2621440
Difícil	$\lceil S/7 + S/13 \rceil$	921826
Naive mejor	$S/4 + 1$	1048577
Naive peor	$S/4 - 1$	1048575

7.2. Comparación con cuatro procesos

El naive puede obtener un speedup muy grande cuando la llave aparece al inicio de un rango. Ese valor describe una posición favorable y no una mejora que se mantenga para cualquier llave. En los casos cercanos al final de un rango, su ventaja disminuye y puede ser menor que uno.

Con cuatro procesos, el speedup del cíclico estuvo entre 2.99 y 3.41 en los cinco casos. El dinámico estuvo entre 1.78 y 2.74. El naive varió entre 0.73 y 1823.98. Estos valores muestran por qué se deben probar varias posiciones de llave.

El reparto cíclico conserva una relación más regular entre el trabajo secuencial y el paralelo. El dinámico evita asignaciones fijas durante toda la búsqueda, aunque pierde un proceso de cálculo al dedicarlo a coordinar. Con cuatro procesos dispone de tres trabajadores, lo que debe considerarse al interpretar la comparación.

El permutado cambia qué llaves aparecen primero. Puede superar al recorrido creciente en una llave y tardar más en otra. Sus resultados se deben leer junto con la desviación estándar, sin afirmar que elimina toda variación.

7.3. Cambio en la cantidad de procesos

La línea ideal muestra una referencia proporcional a la cantidad de procesos. No considera el costo de los mensajes ni el proceso coordinador del dinámico. Los resultados pueden alejarse de esa línea por el reparto de trabajo, los recursos disponibles y la carga del equipo. Se necesitan más repeticiones y mediciones de afinidad para atribuir una diferencia a una causa concreta.

7.4. MPI con OpenMP

Para la llave media, MPI puro con ocho procesos obtuvo un tiempo de 0.2678 segundos y un speedup de 4.50. Las tres configuraciones híbridas con ocho trabajadores tuvieron tiempos entre 0.2845 y 0.2896 segundos. Por tanto, en este caso no se observó una mejora del híbrido frente a MPI puro.

Tabla 3: Tiempo medio, desviación estándar y speedup con cuatro procesos. Los tiempos están en segundos.

Caso	Programa	Media	Desviación	Speedup
Fácil	Cíclico	0.3441	0.0895	3.14
Fácil	Dinámico	0.3934	0.0168	2.74
Fácil	Naive	0.0006	0.0001	1823.98
Fácil	Permutado	0.3835	0.2210	2.82
Fácil	Secuencial	1.0798	0.2473	1.00
Media	Cíclico	0.4039	0.0544	2.99
Media	Dinámico	0.5325	0.0250	2.27
Media	Naive	0.3079	0.0202	3.92
Media	Permutado	0.3222	0.2134	3.74
Media	Secuencial	1.2062	0.0343	1.00
Difícil	Cíclico	0.1333	0.0219	3.34
Difícil	Dinámico	0.1731	0.0071	2.57
Difícil	Naive	0.6058	0.0832	0.73
Difícil	Permutado	0.3036	0.2233	1.47
Difícil	Secuencial	0.4450	0.0364	1.00
Naive mejor	Cíclico	0.1651	0.0094	3.00
Naive mejor	Dinámico	0.2780	0.1494	1.78
Naive mejor	Naive	0.0006	0.0001	799.64
Naive mejor	Permutado	0.2472	0.1799	2.01
Naive mejor	Secuencial	0.4959	0.0272	1.00
Naive peor	Cíclico	0.1451	0.0032	3.41
Naive peor	Dinámico	0.2050	0.0227	2.42
Naive peor	Naive	0.6032	0.0543	0.82
Naive peor	Permutado	0.2810	0.2512	1.76
Naive peor	Secuencial	0.4952	0.0302	1.00

Tabla 4: Versión híbrida para la llave media. Los tiempos están en segundos.

Procesos	Hilos	Media	Desviación	Speedup
1	2	0.7074	0.0482	1.71
1	4	0.4214	0.0354	2.86
1	8	0.2879	0.0390	4.19
2	4	0.2896	0.0738	4.16
4	2	0.2845	0.0349	4.24

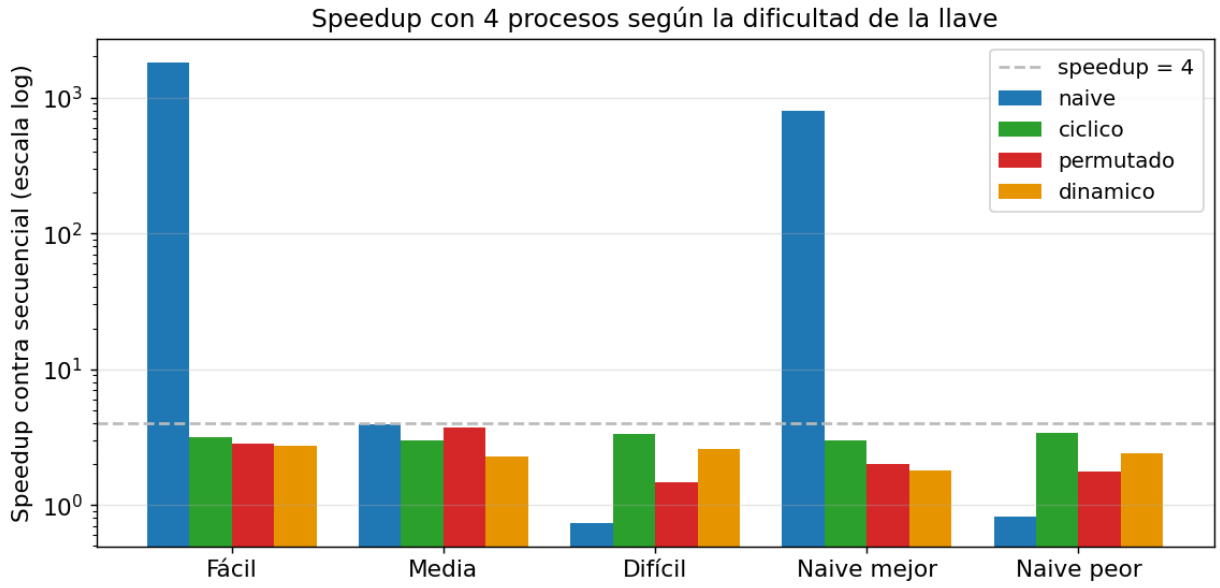


Figura 3: Speedup con cuatro procesos según la posición de la llave. El eje vertical usa escala logarítmica.

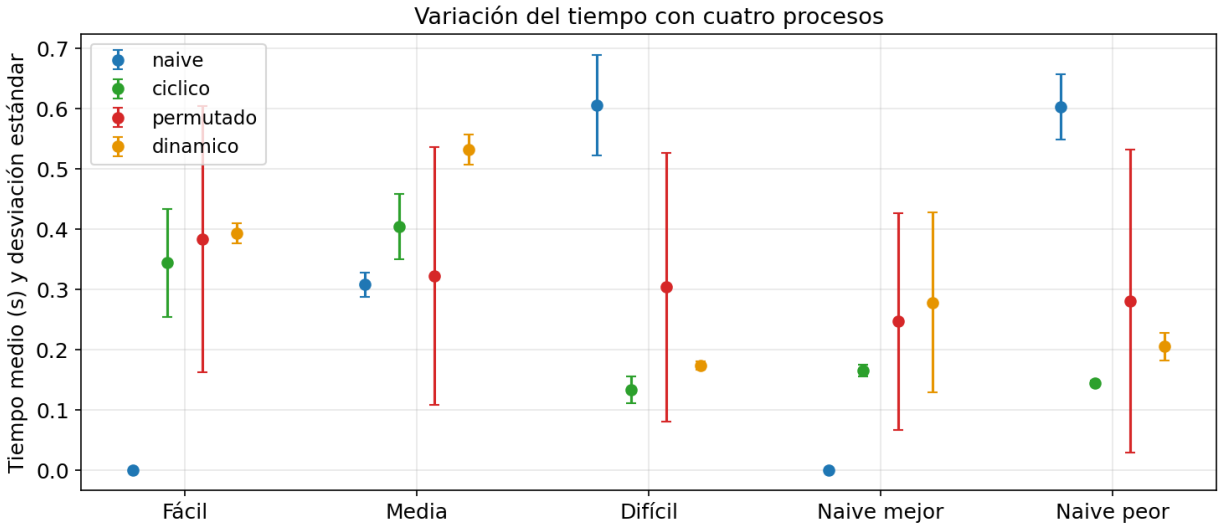


Figura 4: Tiempo medio y desviación estándar entre repeticiones.

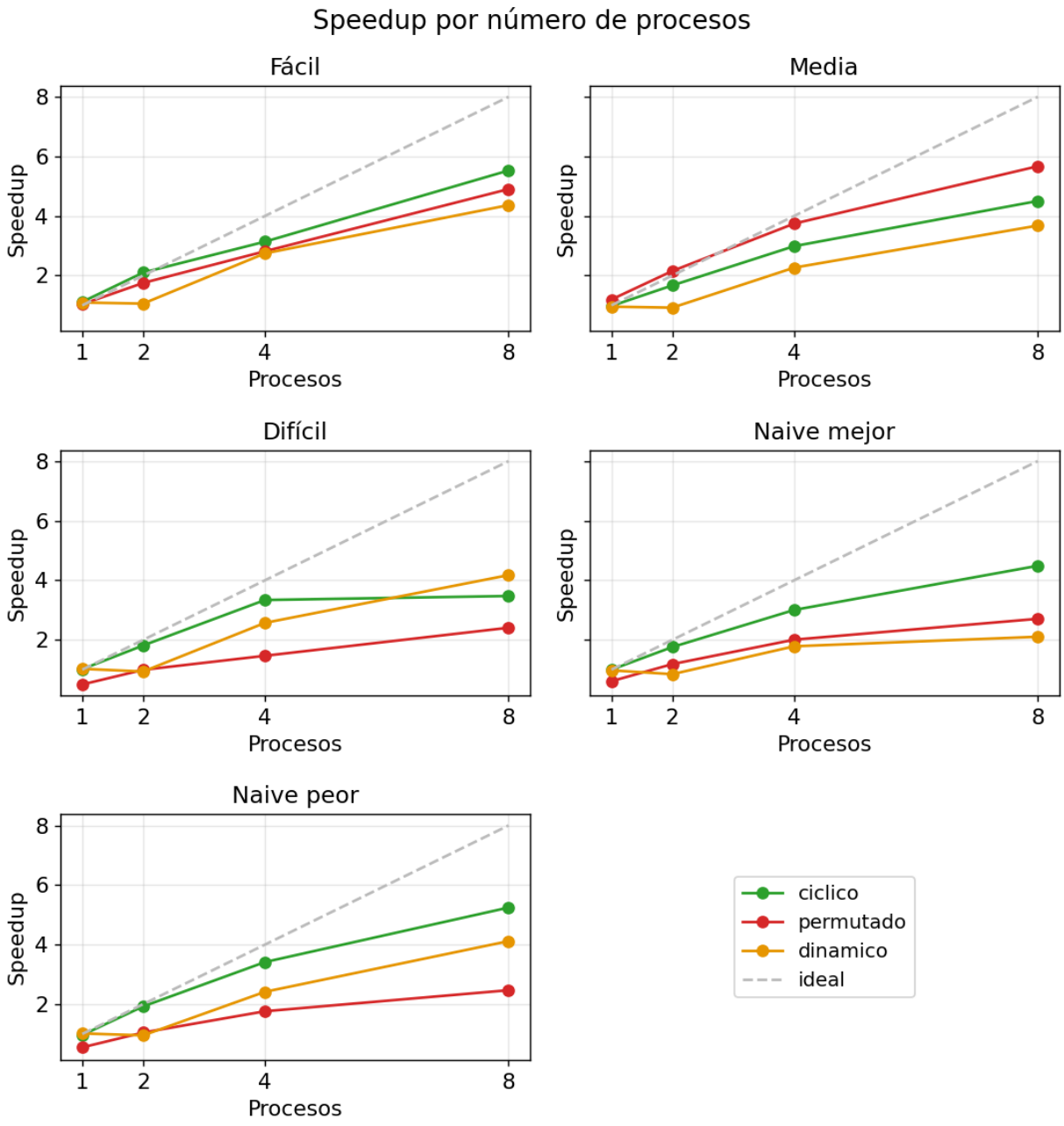


Figura 5: Speedup de las alternativas con distintas cantidades de procesos.

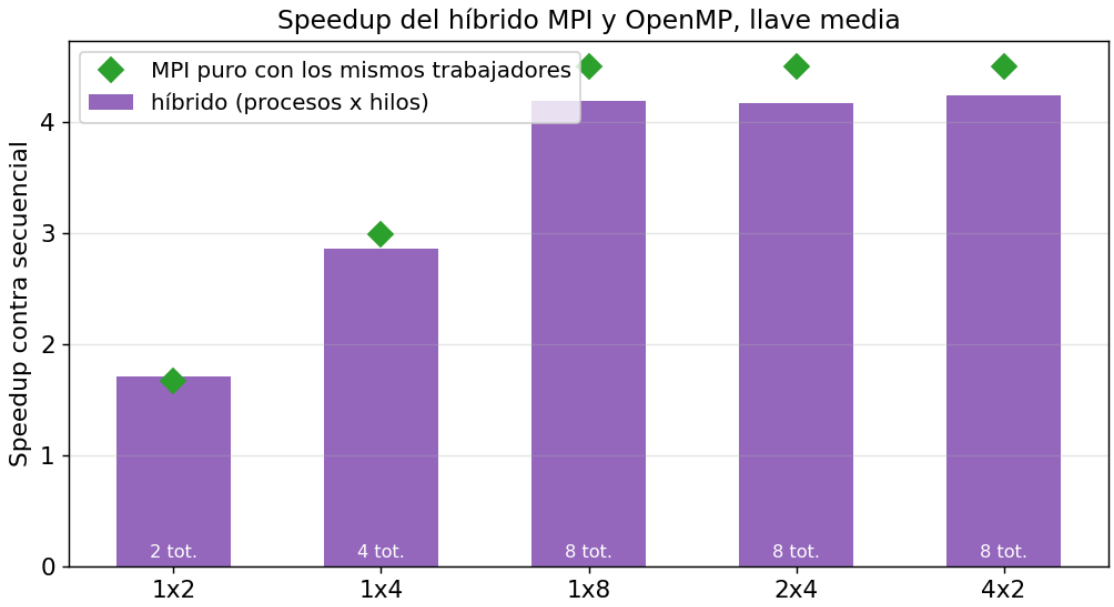


Figura 6: Híbrido frente a MPI puro con igual cantidad total de trabajadores.

Una mejora frente al secuencial no demuestra por sí sola una ventaja de OpenMP frente a MPI. Para evaluar esa ventaja se compara el híbrido con el cíclico de igual cantidad total de trabajadores. El resultado depende de la combinación de procesos e hilos y se limita al equipo y configuración medidos. No se atribuyen diferencias a los núcleos físicos, hilos lógicos o WSL sin pruebas adicionales.

8. Conclusiones

La posición de la llave modifica mucho el tiempo del reparto naive. Una solución al inicio de un rango puede producir un speedup aparente muy alto, mientras otra llave cercana puede obligar a recorrer casi todo el rango. Por eso una sola medición no basta para evaluar el reparto.

La distribución cíclica reparte el avance sobre el espacio y permite una comparación más regular con el recorrido secuencial. El recorrido permutado reduce la relación entre el valor de la llave y su orden de prueba, aunque mantiene variación entre semillas. Ambos cubren el espacio sin repetir llaves entre procesos.

El reparto dinámico permite entregar más bloques a los trabajadores que terminan antes. Su costo incluye los mensajes por bloque y un proceso dedicado a coordinar. El tamaño de bloque limita tanto la frecuencia de pedidos como el trabajo adicional después de un hallazgo.

La coordinación de la salida es tan importante como el reparto. Se deben completar las recepciones y envíos pendientes, incluso cuando no existe una solución o se producen varios hallazgos. En el híbrido, la bandera atómica y los arreglos separados evitan accesos compar-

tidos sin control.

Las pruebas con un espacio reducido permiten comparar estrategias y repetir mediciones. Sus tiempos no deben presentarse como tiempos de búsqueda del espacio completo de 56 bits. El híbrido requiere comparar cantidades iguales de trabajadores antes de afirmar una mejora por el uso de hilos.

9. Recomendaciones

Se recomienda conservar el texto, la frase, la llave y las opciones de compilación cuando se comparan los programas. También conviene registrar los recursos visibles y evitar cambios de carga en el equipo durante una serie de mediciones.

Para el dinámico, se recomienda probar otros tamaños de bloque y medir su efecto antes de cambiar el valor elegido. Para el permutado, conviene ampliar la cantidad de semillas y de llaves para estudiar mejor el tiempo esperado.

Se recomienda usar frases largas y revisar que la llave recuperada coincida con la utilizada al cifrar en las pruebas controladas. En futuras mediciones del híbrido conviene controlar la afinidad y comparar con MPI puro usando los mismos recursos. Una prueba con varias máquinas físicas permitiría estudiar también el costo de la red.

Referencias

[1] National Institute of Standards and Technology. FIPS PUB 46-3, Data Encryption Standard. 1999. https://csrc.nist.gov/pubs/fips/46-3/final.

[2] OpenSSL Project. DES routines. https://docs.openssl.org/3.5/man3/DES_random_key/.

[3] Open MPI Project. Manual de comunicación punto a punto. https://docs.open-mpi.org/en/v5.0.x/man-openmpi/man3/MPI_Irecv.3.html.

[4] OpenMP Architecture Review Board. OpenMP Application Programming Interface, versión 5.2. 2021. https://www.openmp.org/spec-html/5.2/openmp.html.

A. Catálogo de funciones y librerías

Se usan OpenSSL para DES, Open MPI para mensajes, OpenMP para hilos y la biblioteca estándar de C para archivos, memoria, cadenas y operaciones atómicas. El catálogo detalla las entradas, los retornos y los cambios en los datos. Python genera las tablas y gráficas.

Función y archivo	Entradas	Salida y propósito
main (ciclico.c)	int argc, char **argv	Retorno: int. Procesa los argumentos, ejecuta la operación del programa y devuelve su código de salida.
cifrar_archivo (cifrador.c)	const char *ruta_texto, const char *ruta_salida, uint64_t llave	Retorno: int. Carga el texto, lo cifra y escribe el archivo de salida. Devuelve el código de éxito o error.
descifrar_archivo (cifrador.c)	const char *ruta_cifrado, uint64_t llave	Retorno: int. Carga el cifrado y muestra su descifrado. Devuelve el código de éxito o error.
main (cifrador.c)	int argc, char **argv	Retorno: int. Procesa los argumentos, ejecuta la operación del programa y devuelve su código de salida.
es_numero_sin_signo (comun.c)	const char *texto	Retorno: int. Indica si la cadena contiene solo dígitos y no está vacía.
validar_frase (comun.c)	const char *frase, size_t longitud_cifrado	Retorno: int. Comprueba que la frase no esté vacía ni sea mayor que el cifrado. Advierte si es corta.
leer_llave (comun.c)	const char *texto, uint64_t *llave	Retorno: int. Escribe la llave validada en el puntero de salida. Devuelve 1 si está entre 0 y 2 elevado a 56 menos 1.
leer_bits (comun.c)	const char *texto, int *bits	Retorno: int. Escribe en bits un valor entre 1 y 56. Devuelve 1 si es válido.
leer_archivo (comun.c)	const char *ruta, size_t *longitud	Retorno: unsigned char *. Devuelve memoria reservada con el archivo y escribe su longitud. Devuelve NULL si falla. El llamador libera el arreglo.
busqueda_preparar (comun.c)	const char *ruta_cifrado, const char *frase, const char *texto_bits, Busqueda *busqueda	Retorno: int. Valida las entradas y completa la estructura de búsqueda. Devuelve 1 si queda lista o 0 si falla.
busqueda_liberar (comun.c)	Busqueda *busqueda	Retorno: void. Libera los arreglos de la búsqueda y deja sus punteros en NULL.
imprimir_datos_busqueda (comun.c)	const Busqueda *busqueda, const char *programa	Retorno: void. Muestra programa, archivo, frase, procesos y espacio de búsqueda.
imprimir_llave (comun.c)	const Busqueda *busqueda, int encontrada, uint64_t llave	Retorno: void. Muestra la llave y el texto descifrado, o informa que no se encontró.

Función y archivo	Entradas	Salida y propósito
imprimir_tiempo (comun.c)	double segundos, uint64_t iteraciones	Retorno: void. Muestra segundos y velocidad calculada con la cantidad total de intentos.
imprimir_texto (comun.c)	const unsigned char *texto	Retorno: void. Muestra la cadena y sustituye caracteres de control para conservar la salida de terminal.
imprimir_hexadecimal (comun.c)	const unsigned char *datos, size_t longitud	Retorno: void. Muestra los bytes en formato hexadecimal.
segundos_ahora (comun.c)	void	Retorno: double. Devuelve el tiempo de un reloj monótono en segundos.
expandir_llave (des.c)	uint64_t llave, DES_cblock *llave_des	Retorno: void. Escribe en llave_des los ocho bytes de llave DES con paridad impar.
transformar (des.c)	uint64_t llave, unsigned char *datos, size_t longitud, int operacion	Retorno: void. Modifica los datos con DES ECB según la operación de cifrado o descifrado. La longitud debe ser múltiplo de ocho.
des_longitud_rellena (des.c)	size_t longitud	Retorno: size_t. Devuelve la longitud redondeada al siguiente múltiplo de ocho.
des_cifrar (des.c)	uint64_t llave, unsigned char *datos, size_t longitud	Retorno: void. Sobrescribe los datos con su cifrado DES ECB.
des_descifrar (des.c)	uint64_t llave, unsigned char *datos, size_t longitud	Retorno: void. Sobrescribe los datos con su descifrado DES ECB.
des_probar_llave (des.c)	uint64_t llave, const unsigned char *cifrado, size_t longitud, const char *frase, unsigned char *trabajo	Retorno: int. Escribe el descifrado en trabajo. Devuelve 1 cuando contiene la frase, o 0 en otro caso. Requiere longitud más uno bytes de trabajo.
coordinar (dinamico.c)	const Busqueda *busqueda, uint64_t bloque	Retorno: void. Responde pedidos y reparte rangos sin repetir. Retira a todos los trabajadores al agotarse el espacio o recibir un hallazgo.
trabajar (dinamico.c)	Busqueda *busqueda, ResultadoLocal *local	Retorno: void. Solicita y recorre bloques. Actualiza en local el hallazgo, su llave y el número de intentos.
main (dinamico.c)	int argc, char **argv	Retorno: int. Procesa los argumentos, ejecuta la operación del programa y devuelve su código de salida.
reservar_trabajos (hibrido.c)	int hilos, size_t bytes_trabajo	Retorno: unsigned char **. Devuelve un arreglo de buffers, uno por hilo. Devuelve NULL si falla y libera las reservas parciales.
liberar_trabajos (hibrido.c)	unsigned char **trabajos, int hilos	Retorno: void. Libera los buffers y el arreglo de punteros de los hilos.

Función y archivo	Entradas	Salida y propósito
main (hibrido.c)	int argc, char **argv	Retorno: int. Procesa los argumentos, ejecuta la operación del programa y devuelve su código de salida.
reservar_memoria (mpi_comun.c)	Busqueda *busqueda, size_t bytes_frase	Retorno: int. Reserva cifrado, trabajo y frase para un proceso. Devuelve 1 si todas las reservas tienen éxito.
aviso_cerrar (mpi_comun.c)	Aviso *aviso, int avisos_esperados	Retorno: void. Completa o cancela la recepción y recibe los avisos restantes según los hallazgos acordados.
mpi_preparar_busqueda (mpi_comun.c)	int *argc, char ***argv, const char *uso, int argumentos_extra, Busqueda *busqueda	Retorno: int. Inicia MPI, verifica soporte de hilos y reparte entradas. Completa busqueda y puede modificar argc y argv. Devuelve 1 si se puede buscar.
mpi_imprimir_datos (mpi_comun.c)	const Busqueda *busqueda, const char *programa	Retorno: void. Muestra los datos de búsqueda solo desde el proceso 0.
aviso_abrir (mpi_comun.c)	Aviso *aviso	Retorno: void. Inicializa aviso y abre una recepción no bloqueante de la llave.
aviso_llego (mpi_comun.c)	Aviso *aviso	Retorno: int. Actualiza la solicitud de aviso. Devuelve 1 si se completó la recepción.
aviso_enviar (mpi_comun.c)	const Busqueda *busqueda, Aviso *aviso, uint64_t llave	Retorno: void. Guarda la llave en aviso e inicia los envíos no bloqueantes a los demás procesos. Conserva solicitudes hasta el cierre.
mpi_iniciar_cronometro (mpi_comun.c)	void	Retorno: double. Ejecuta una barrera MPI explícita y devuelve el instante de inicio.
mpi_cerrar_busqueda (mpi_comun.c)	Busqueda *busqueda, Aviso *aviso, const ResultadoLocal *local	Retorno: void. Completa mensajes, reúne llave y métricas, imprime resultados, libera memoria y finaliza MPI.
main (naive.c)	int argc, char **argv	Retorno: int. Procesa los argumentos, ejecuta la operación del programa y devuelve su código de salida.
mezclar (permutado.c)	uint64_t valor	Retorno: uint64_t. Devuelve un entero con los bits mezclados para obtener los parámetros de la permutación.
leer_semilla (permutado.c)	const char *texto, uint64_t *semilla	Retorno: int. Escribe una semilla sin signo de 64 bits. Devuelve 1 si es válida o 0 si falla.
main (permutado.c)	int argc, char **argv	Retorno: int. Procesa los argumentos, ejecuta la operación del programa y devuelve su código de salida.
main (secuencial.c)	int argc, char **argv	Retorno: int. Procesa los argumentos, ejecuta la operación del programa y devuelve su código de salida.

B. Bitácora y capturas de ejecución

La bitácora individual se encuentra en resultados/bitacora.csv y el resumen en resultados/resumen.csv. Cada corrida conserva su salida en resultados/ejecuciones/. Los registros de compilación, entorno, validación, OpenSSL, pruebas de la guía y tamaños secuenciales se encuentran en resultados/. Las capturas muestran esos registros de ejecuciones reales abiertos en el editor de texto.

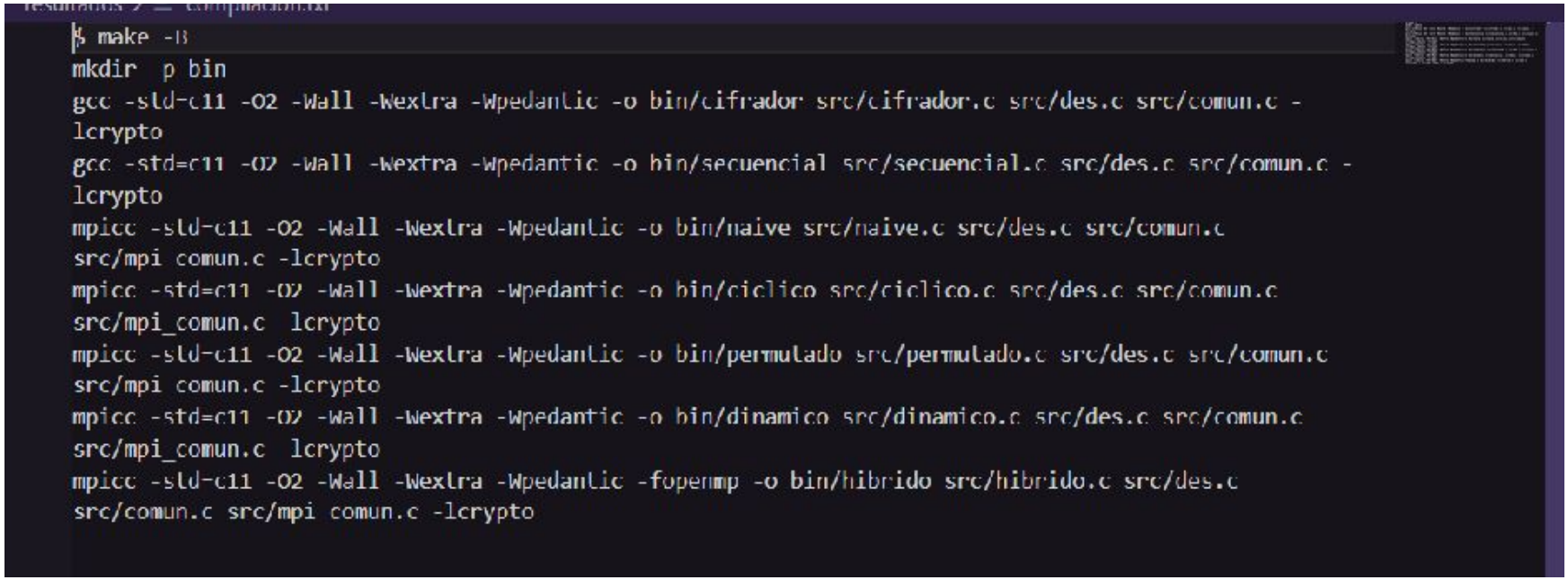


Figura 7: Registro de compilación de los siete programas.

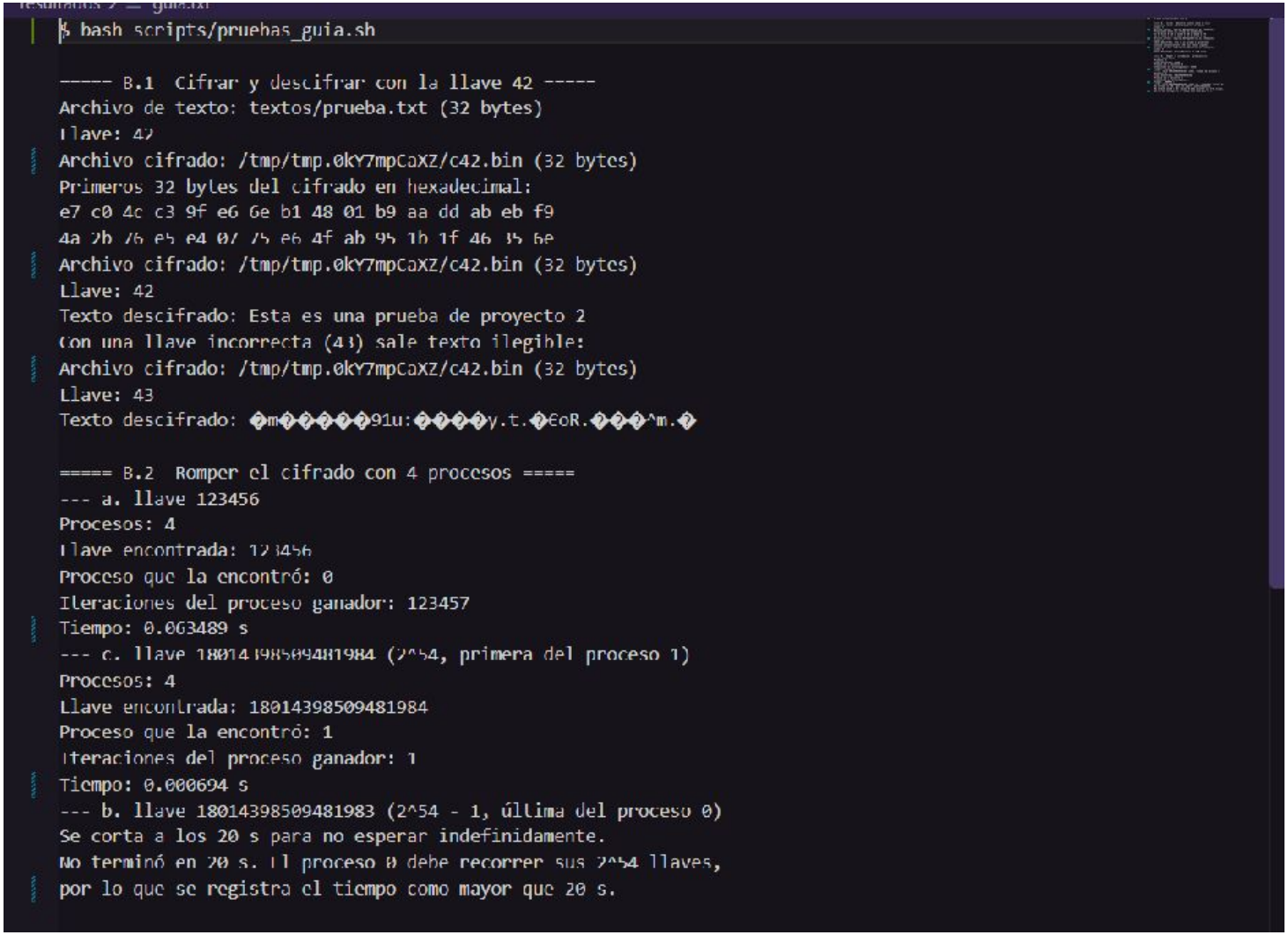


Figura 8: Cifrado y descifrado con la llave 42 y pruebas de posición con cuatro procesos.

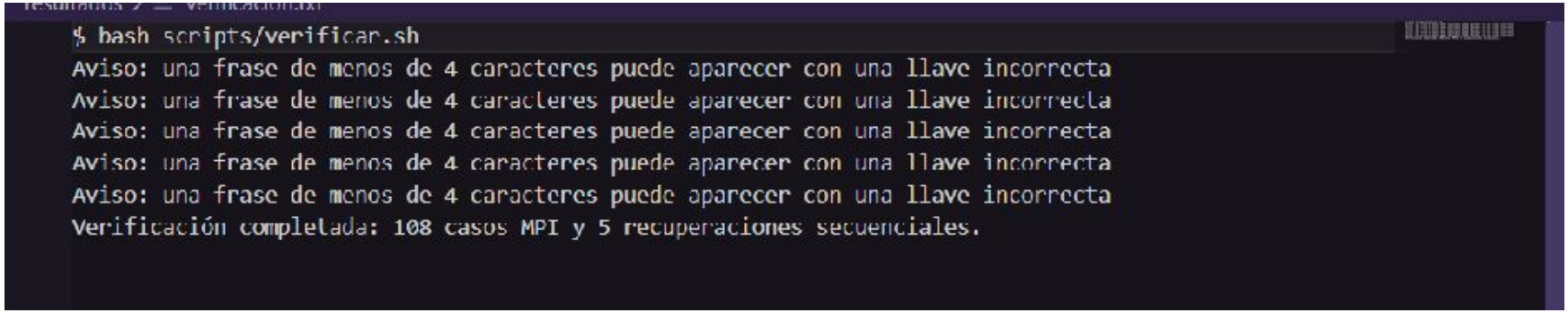


Figura 9: Verificación de recuperación, agotamiento del espacio y validación de entradas.

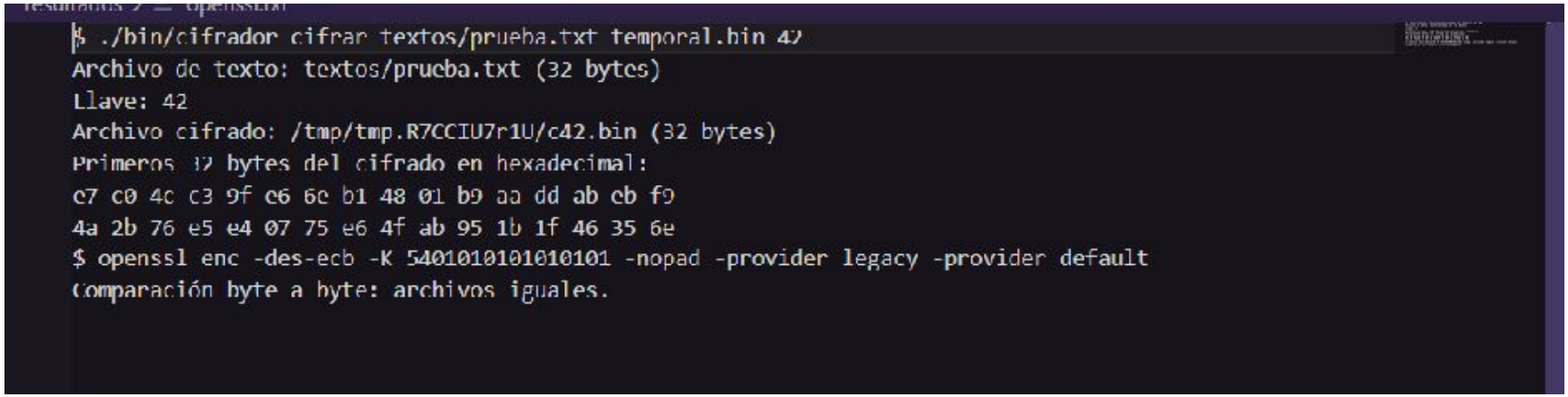


Figura 10: Comparación del cifrado con la herramienta OpenSSL.

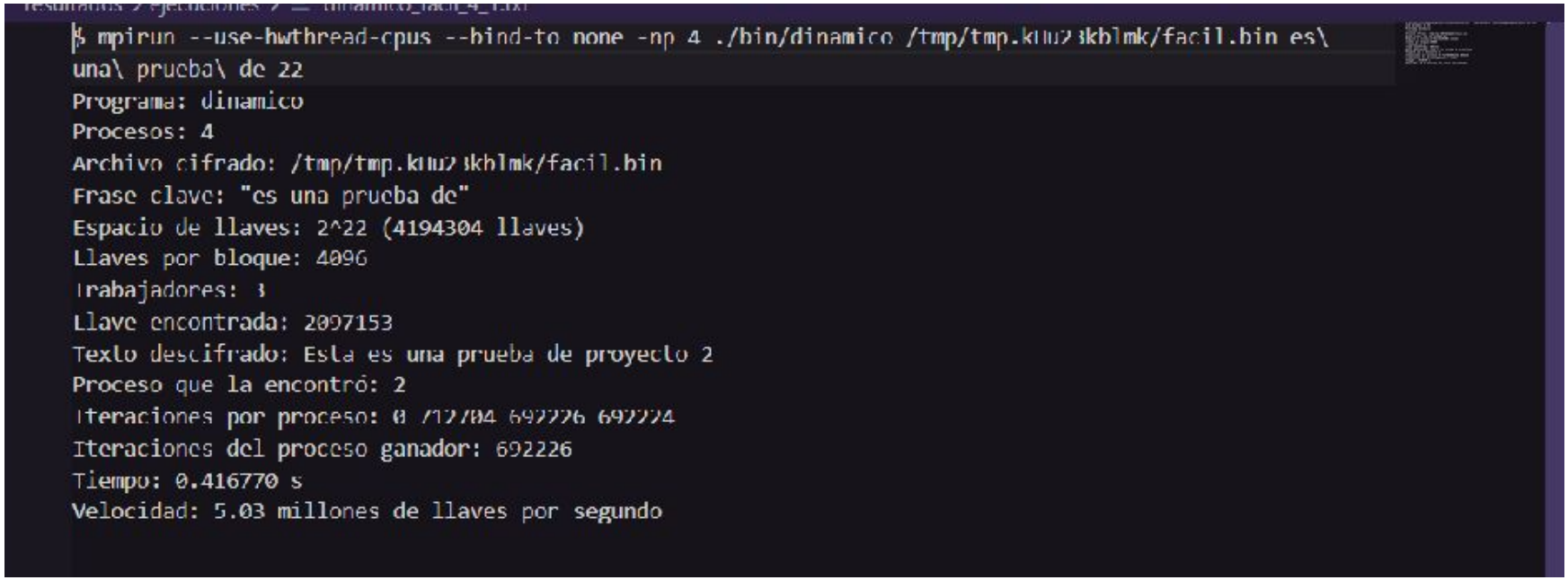


Figura 11: Recuperación de una llave mediante bloques dinámicos.

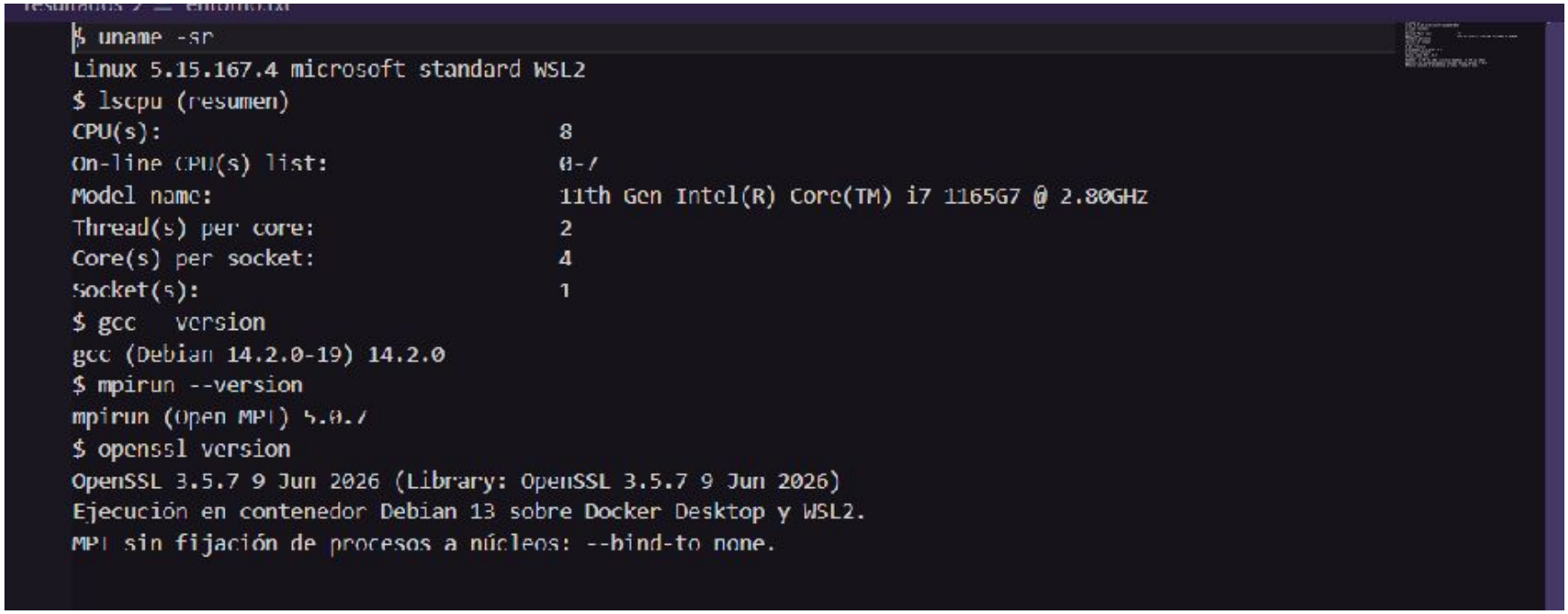


Figura 12: Entorno de ejecución y versiones de las herramientas.